Settings Page Info

Page Overview

Purpose

The Settings page is the central configuration surface for `resmon`. It groups all user-configurable behavior into seven sub-tabs — Email, Cloud Account, Cloud Storage, AI, Storage, Notifications, and Advanced — each handling a self-contained slice of backend configuration, secret management, or diagnostics. (The previous **About App** sub-tab was relocated to the new top-level About resmon page in Update 3.)

Primary User Flows

1. Open **Settings** from the sidebar. The page renders a tab strip and a panel area; the route defaults to the Email panel.
2. Click a tab to navigate to its panel (`/settings/email` , `/settings/account` , `/settings/cloud` , `/settings/ai` , `/settings/storage` , `/settings/notifications` , `/settings/advanced`).
3. Within a panel, edit fields and press **Save** (or the panel-specific action button) to persist the change through the corresponding `PUT /api/settings/*` endpoint (or a credential / service endpoint, depending on the tab).
4. Test actions (send test email, test API key, list models, link Google Drive, install service, refresh scheduler jobs) call their backend endpoints directly and surface an inline status line.

Inputs and Outputs

- **Inputs** — per-panel form fields, toggles, and action buttons; secrets typed into password inputs (SMTP password, AI provider API key).
- **Outputs** — rows written to the `settings` table via `set_setting` , OS-keychain entries written through `credential_manager.store_credential` , OAuth tokens acquired by `cloud_storage.authorize_google_drive` , and scheduler / service diagnostic data surfaced read-only.

Known Constraints

- Secrets (SMTP password, AI provider API keys) are never returned to the frontend; the UI only sees presence booleans from `GET /api/credentials` .
- The **Cloud Account** tab is an "under construction" placeholder in this build — the backend routes (IMPL-27..40) exist, but no hosted identity provider is wired, so sign-in is not available.
- The **AI** tab's Custom provider branch disables Save unless the base URL is HTTPS or the host is loopback (`localhost` , `127.0.0.1` , `::1`), per IMPL-AI12.
- The PDF / TXT retention policies on the **Storage** tab are reserved for a future per-paper artifact download feature and have no effect on current Deep Dive / Deep Sweep output.

Frontend

Route and Main Component

- **Route:** `/settings` (nested routes under `/settings/<tab>`).
- **Main component:** `SettingsPage` (resmon_scripts/frontend/src/pages/SettingsPage.tsx).
- The page renders an `<h1>Settings</h1>` header, a `.settings-nav` tab strip of `NavLink` elements, and a nested `<Routes>` block. The index route redirects to `/settings/email` via `<Navigate to="email" replace />` .

Child Components and Hooks

- Panel components (all under `resmon_scripts/frontend/src/components/Settings/`): `EmailSettings` , `CloudAccountSettings` , `CloudSettings` , `AISettings` , `StorageSettings` , `NotificationSettings` , `AdvancedSettings` . (The previous `AboutAppSettings` was relocated to `components/AboutResmon/AboutAppTab.tsx` and is now mounted on the About resmon page.)
- Per-page tutorial deep-link: a `TutorialLinkButton` (`anchor="settings"`) is rendered in `<div className="page-header">` next to the `<h1>Settings</h1>` title, and each panel header renders its own `TutorialLinkButton` (`settings-email` , `settings-account` , `settings-cloud` , `settings-ai` , `settings-storage` , `settings-notifications` , `settings-advanced`) so users can jump from any sub-tab to the matching tutorial section. See about_resmon_info.md.

- Shared help component: `PageHelp` (each panel wires its own `storageKey`, e.g. `settings-email`, `settings-ai`, `settings-advanced`).
- Hooks: `useState`, `useEffect`, `useCallback` for panel-local state; `NavLink` / `Navigate` / `Routes` from `react-router-dom` for tab routing; `apiClient` wrapper for REST calls.

UI State Model

`SettingsPage` itself holds no state. Each panel owns its own local state; common patterns:

- `loading: boolean` / `saving: boolean` — request guards.
- `status: string` — transient inline status line, cleared via `setTimeout`.
- A `settings` object mirroring the tab's settings subset (Email, AI, Storage, Notifications, Cloud Storage).
- Secret inputs (SMTP password, AI key) held in separate state with a companion `*Masked` flag; presence of the stored credential fetched as a boolean.

Key Interactions and Events

- **Tab switch** — `NavLink` updates the URL; the matching panel mounts and triggers its own initial `GET`.
- **Save** — each panel's primary button calls `apiClient.put('/api/settings/<slice>', { settings })` and reloads the presence / status it needs.
- **Test** — panel-specific side actions: `POST /api/settings/email/test`, `POST /api/credentials/validate`, `POST /api/ai/models`, `POST /api/cloud/backup`, `POST /api/service/install`.
- **Credential write / delete** — `PUT /api/credentials/{name}` and `DELETE /api/credentials/{name}` from Email and AI panels.

Error and Empty States

- Each panel shows a muted `Loading…` placeholder until its initial fetch resolves.
- Inline `status` strings surface both successes and errors; error strings are prefixed `Error:` / `Test failed` and auto-clear after a few seconds.
- The Cloud Account panel shows a fixed "Under construction" block instead of any functional control.
- Advanced shows `jobs=[]` with an error line if `GET /api/scheduler/jobs` fails, rather than hiding the diagnostics card.

Panel: Email

- Component: `EmailSettings` (resmon_scripts/frontend/src/components/Settings/EmailSettings.tsx).
- Fields: `smtp_server`, `smtp_port`, `smtp_username`, `smtp_from`, `smtp_to` — persisted through `PUT /api/settings/email`.
- SMTP password is stored in the OS keychain under the credential name `smtp_password` via `PUT /api/credentials/smtp_password`; the UI fetches only a presence boolean from `GET /api/credentials`. On store, whitespace is stripped from the input so Gmail App Passwords (displayed as four space-separated groups) become the raw 16-character secret.
- Actions: **Save**, **Store password**, **Remove password** (`DELETE /api/credentials/smtp_password`), **Send test email** (`POST /api/settings/email/test`).

Panel: Cloud Account

- Component: `CloudAccountSettings` (resmon_scripts/frontend/src/components/Settings/CloudAccountSettings.tsx).
- Renders a `PageHelp` block describing why cloud sign-in matters (cloud routines, cloud-scoped credentials, cloud-executed reports) and a privacy note that local executions never depend on cloud sign-in.
- Body is a static "Under construction" placeholder; no sign-in control is rendered because no hosted identity provider is wired in this build. The underlying backend routes (`/api/cloud-auth/*`) exist and are covered by the Backend section below.

Panel: Cloud Storage

- Component: `CloudSettings` (resmon_scripts/frontend/src/components/Settings/CloudSettings.tsx).
- State: `isLinked`, `apiOk`, `apiReason`, `autoBackup` — loaded from `GET /api/cloud/status` and `GET /api/settings/cloud`.
- Actions: **Link Google Drive** (`POST /api/cloud/link` — triggers the OAuth installed-app flow in the backend), **Unlink** (`POST /api/cloud/unlink`), **Toggle auto-backup** (writes `cloud_auto_backup` via `PUT /api/settings/cloud`), **Back up now** (`POST /api/cloud/backup` — returns `{ uploaded, total_files, folder_name, web_view_link }`).
- Surfaces Drive-API error reasons via `API_REASON_HINTS` (`accessNotConfigured`, `insufficientPermissions`, `no_token`) so the user can resolve Google Cloud Console / scope-consent issues.

Panel: AI

- Component: `AISettings` (resmon_scripts/frontend/src/components/Settings/AISettings.tsx).
- Provider whitelist (IMPL-AI9 / AI10): `anthropic` , `openai` , `google` , `xai` , `meta` , `deepseek` , `alibaba` , `local` , `custom` . Each provider has a suggested model placeholder (e.g. `gpt-4o-mini` , `claude-3-5-haiku-latest` , `gemini-2.5-flash` , `grok-4` , `meta-llama/Llama-3.3-70B-Instruct-Turbo` , `deepseek-chat` , `qwen-plus` , `llama3`).
- Settings keys persisted through `PUT /api/settings/ai` : `ai_provider` , `ai_model` , `ai_local_model` , `ai_summary_length` , `ai_tone` , `ai_temperature` , `ai_extraction_goals` , `ai_custom_base_url` , `ai_custom_header_prefix` , and `ai_default_models` (a `{provider: model_id}` map of per-provider default models persisted as a JSON-encoded settings entry; included in the settings allowlist enforced by the backend). IMPL-AI11 separated `ai_local_model` from `ai_tone` and introduced a one-shot migration heuristic (`looksLikeModelId`) that moves a misplaced model id out of `ai_tone` .
- **Stored API Keys table.** A four-column table (`Provider` , `Key Name` , `Status` , `Actions`) lists every provider for which `GET /api/credentials` reports a stored key. Clicking a row's provider name switches the panel's Provider control to that provider so the user can edit its model / temperature / tone in place. The currently-active provider's row is annotated with a `Default` badge. The Actions column exposes per-row `Clear default model` (deletes the matching key from `ai_default_models` via `PUT /api/settings/ai`) and `Clear API key` (issues `DELETE /api/credentials/{name}` for that provider). Writers serialize through a `useRef` -backed inflight latch so concurrent toggles never race.
- **Per-provider keys (multi-provider).** Each provider has its own keyring entry (`openai_api_key` , `anthropic_api_key` , `google_api_key` , `xai_api_key` , `meta_api_key` , `deepseek_api_key` , `alibaba_api_key` , `local_api_key` , `custom_llm_api_key`); switching the Provider control no longer clobbers another provider's key. On first run the backend transparently migrates any legacy single-key `ai_api_key` entry into the slot for the currently-selected provider (idempotent, runs once per process).
- **InfoTooltip annotations.** The four primary AI labels (`Summary length` , `Tone` , `Temperature` , `Extraction goals`) carry `InfoTooltip` icons that explain (1) what the field controls, (2) the value range / valid options, and (3) that per-execution overrides on Deep Dive / Deep Sweep / Routines transparently override these defaults via per-field merge.
- Custom-provider UX guard (IMPL-AI12): `validateCustomBaseUrl` disables Save if the base URL is not HTTPS unless the host is `localhost` , `127.0.0.1` , or `::1` . The backend `llm_factory` enforces the same rule as a hard check.
- Credential write: the provider's API key is stored via `PUT /api/credentials/{name}` where `name` is derived from the provider (e.g. `openai_api_key` , `anthropic_api_key` , `custom_llm_api_key`). The backend responds with presence only.
- Actions: **Save** (`PUT /api/settings/ai`), **Test key** (`POST /api/credentials/validate`), **Load models** (`POST /api/ai/models` — returns the per-provider model list using either the freshly typed key or the stored credential), **Save as default model** (writes the chosen `ai_model` into `ai_default_models[provider]` via `PUT /api/settings/ai` so the per-provider default survives a provider switch).

Panel: Storage

- Component: `StorageSettings` (resmon_scripts/frontend/src/components/Settings/StorageSettings.tsx).
- Fields: `pdf_policy` , `txt_policy` , `archive_after_days` , `export_directory` — persisted through `PUT /api/settings/storage` . Policy values are constrained to `save` / `archive` / `discard` .
- `export_directory` is the active setting used by configuration / execution exports. The PDF and TXT policy controls are marked in the UI as reserved for a future per-paper artifact download feature.
- Help copy calls out that retention policy prunes reports older than the window on daemon startup, and that the on-disk cloud-execution cache (`CLOUD_CACHE_MAX_BYTES_DEFAULT`) is capped independently.

Panel: Notifications

- Component: `NotificationSettings` (resmon_scripts/frontend/src/components/Settings/NotificationSettings.tsx).
- Fields: `notify_manual: boolean` and `notify_automatic_mode: 'all' | 'selected' | 'none'` — loaded from `GET /api/settings/notifications` and saved with `PUT /api/settings/notifications` .
- Also surfaces the browser-level desktop-notification permission state and a **Request permission** button that calls `Notification.requestPermission()` in the renderer.
- **Cross-platform desktop notifications.** Routine and manual completions raise a native OS notification on macOS (NSUserNotificationCenter / UNUserNotificationCenter via Electron), Linux (libnotify / `notify-send` via the dispatcher fallback), and Windows (Toast notifications via the dispatcher fallback). The dispatcher is invoked from both the foreground app and the headless `resmon-daemon` , so notifications fire even when the Electron UI is closed; a stale daemon started before this update silently drops the new code path until restarted.
- Help text clarifies that email notifications and cloud uploads are independent of this tab.

Panel: Advanced

- Component: `AdvancedSettings` (resmon_scripts/frontend/src/components/Settings/AdvancedSettings.tsx).

- **Background daemon** — `GET /api/service/status` returns `{ installed, unit_path, platform }`; `POST /api/service/install` and `POST /api/service/uninstall` toggle the OS service unit (IMPL-26). Health polling of `/api/health` at a 5-second interval surfaces the renderer-attached backend's `{ status, pid, started_at, version }`. **Update 4 (`5_5_26`):** the panel additionally polls `GET /api/service/daemon-status`, a backend route that reads `daemon.lock` and probes the daemon's actual port directly so the displayed pid / version / last-started reflect the *real* daemon rather than whichever backend the renderer happens to be attached to. The status block renders three explicit states ("daemon up" with an `, this process` tag when the lock points at the current backend; "lock present but unreachable" with the diagnostic error; "no daemon running"), and the renderer-attached backend's identity is shown separately as a `· this window → pid …` diagnostic line so any divergence is immediately visible.
- **Concurrent executions (IMPL-R12)** — `GET /api/settings/execution` returns `{ max_concurrent_executions, routine_fire_queue_limit }`; the panel persists edits through `PUT /api/settings/execution`. Limits flow into `admission` (IMPL-R1 / R2) and into the scheduler's routine-fire queue (IMPL-R3 / R6).
- **Scheduler diagnostics** — `GET /api/scheduler/jobs` returns `{ id, name, next_run_time, trigger }` entries from the APScheduler job store. A Refresh action re-fetches the list.
- **Danger Zone (Update 3 / `4_27_26`)** — appended at the bottom of the Advanced panel. Two columns are rendered side by side: `Local device` (active) and `Cloud account` (scaffolding, all buttons rendered disabled with a "Coming soon — requires cloud sign-in" tooltip). Each column exposes the same eight destructive actions:
  i. **Erase all AI API keys** — `POST /api/admin/erase-ai-keys`. Deletes every credential in `AI_CREDENTIAL_NAMES` from the OS keyring. Simple green `Confirm` / red `Cancel` modal; no typed gate.
  ii. **Erase all repo API keys** — `POST /api/admin/erase-repo-keys`. Deletes every credential in `catalog_credential_names()` from the OS keyring. Simple confirm modal; no typed gate.
  iii. **Erase all configs** — `POST /api/admin/erase-configs`. Deletes every row in `saved_configurations`; cascades to any routine linked through `linked_routine_id`. Typed- `CONFIRM` gate (case-sensitive, all caps).
  iv. **Erase execution history** — `POST /api/admin/erase-executions`. Deletes every row in `executions` (and child rows: progress events, logs links) and resets the `sqlite_sequence` row for `executions` so the next run starts at `Execution #1`. Typed- `CONFIRM` gate.
  v. **Erase all execution data** — `POST /api/admin/erase-execution-data`. Composite of (3) + (4). Typed- `CONFIRM` gate.
  vi. **Erase all app data** — `POST /api/admin/erase-app-data`. Composite of (1) + (2) + (5); non-AI Settings tabs (Email, Cloud Storage, Storage, Notifications, Advanced) are preserved. Typed- `CONFIRM` gate.
  vii. **Reset all settings** — `POST /api/admin/reset-settings`. Resets every Settings tab to defaults, deletes every API key (AI + repo + SMTP password), and clears the cached cloud-account email; configs and executions are preserved. Typed- `CONFIRM` gate.
  viii. **Factory reset** — `POST /api/admin/factory-reset`. Composite of (6) + (7). Typed- `CONFIRM` gate.
- The two confirmation modals are rendered locally inside `AdvancedSettings.tsx`. The simple variant (buttons 1–2) shows a green Confirm + red Cancel pair. The typed variant (buttons 3–8) renders the action's irreversibility warning, an `<input>` whose value must equal the literal string `CONFIRM` exactly (case-sensitive, no surrounding whitespace), and a red Confirm button that stays `disabled` until the input matches. Both modals close on backdrop click only when the action is not in flight.
- On success, the panel broadcasts on `configurationsBus` and `routinesBus`, and dispatches a synthetic `resmon:execution-completed` window event so the Dashboard, Configurations, Routines, Calendar, and Results & Logs surfaces refetch immediately.
- Cloud-column buttons share the same `LOCAL_DANGER_ACTIONS` definition with the endpoint string rewritten to the `/api/admin/cloud/...` mirror; until the cloud-account feature lands, the column is rendered disabled at the UI layer.

Backend

API Endpoints

Method	Path	Purpose	Pa
GET	`/api/health`	Daemon status `{ status, pid, started_at, version }`	Advan
GET	`/api/credentials`	Presence map for allowed credential names (no values)	Email,
PUT	`/api/credentials/{key_name}`	Store a secret via OS keyring	Email,
DELETE	`/api/credentials/{key_name}`	Remove a stored secret	Email,
POST	`/api/credentials/validate`	Probe an API key against its provider endpoint	AI
GET / PUT	`/api/settings/email`	SMTP configuration row set	Email
POST	`/api/settings/email/test`	Send a test email using the current config + stored password	Email

Method	Path	Purpose	Pa
GET / PUT	`/api/settings/ai`	AI provider / model / tone / temperature / extraction goals / custom-base-url settings	AI
POST	`/api/ai/models`	List models for a provider using ephemeral or stored key	AI
GET / PUT	`/api/settings/cloud`	Cloud-storage preferences (e.g. `cloud_auto_backup`)	Cloud Storag
POST	`/api/cloud/link`	Start Google Drive OAuth flow	Cloud Storag
POST	`/api/cloud/unlink`	Revoke and drop the Drive token	Cloud Storag
GET	`/api/cloud/status`	`{ is_linked, api_ok, api_reason }`	Cloud Storag
POST	`/api/cloud/backup`	Upload `resmon_reports/` to Drive; returns `{ uploaded, total_files, folder_name, web_view_link }`	Cloud Storag
GET / PUT	`/api/settings/storage`	PDF / TXT policies, archive retention, export directory	Storag
GET / PUT	`/api/settings/notifications`	`notify_manual` , `notify_automatic_mode`	Notific
GET	`/api/service/status`	OS service-unit state	Advan
GET	`/api/service/daemon-status`	Ground-truth daemon status read from `daemon.lock` and a live health probe of its port (Update 4 / Fix E)	Advan
POST	`/api/service/install`	Install platform-specific service unit	Advan
POST	`/api/service/uninstall`	Remove platform-specific service unit	Advan
GET / PUT	`/api/settings/execution`	`max_concurrent_executions` , `routine_fire_queue_limit`	Advan
GET	`/api/scheduler/jobs`	APScheduler job listing	Advan
POST	`/api/admin/erase-ai-keys`	Erase every saved AI provider API key from the OS keyring (Danger Zone)	Advan
POST	`/api/admin/erase-repo-keys`	Erase every saved research-repository API key from the OS keyring (Danger Zone)	Advan
POST	`/api/admin/erase-configs`	Delete every saved configuration and cascade to linked routines (Danger Zone)	Advan
POST	`/api/admin/erase-executions`	Delete every execution row and reset the `Execution #N` counter (Danger Zone)	Advan
POST	`/api/admin/erase-execution-data`	Composite: erase configs + erase executions (Danger Zone)	Advan
POST	`/api/admin/erase-app-data`	Composite: erase AI keys + repo keys + execution data; non-AI settings preserved (Danger Zone)	Advan
POST	`/api/admin/reset-settings`	Reset every Settings tab + erase AI / repo / SMTP keys; configs and executions preserved (Danger Zone)	Advan
POST	`/api/admin/factory-reset`	Composite: erase app data + reset settings (Danger Zone)	Advan
POST	`/api/admin/cloud/{erase-*,reset-settings,factory-reset}`	Cloud-account mirrors of the eight Danger Zone actions; scaffolding only — disabled in the UI until cloud sign-in lands (Danger Zone)	Advan

Method	Path	Purpose	Pa
POST / GET / DELETE / PUT	`/api/cloud-auth/session` , `/api/cloud-auth/status` , `/api/cloud-auth/refresh` , `/api/cloud-auth/sync`	Cloud account session plumbing (dormant UI in this build)	Cloud Accou

Request / Response Patterns

- Settings reads (`GET /api/settings/<slice>`) return a flat `{ key: value }` mapping derived from `get_setting(conn, key)` calls in resmon_scripts/resmon.py.
- Settings writes use the envelope `PUT /api/settings/<slice> { "settings": { key: value, ... } }` ; the handler validates allowed keys for that slice and dispatches `set_setting(conn, key, value)` per entry.
- Credential routes never echo values. `GET /api/credentials` returns a `{ name: { present: bool } }` map constructed from the union of `AI_CREDENTIAL_NAMES ∪ SMTP_CREDENTIAL_NAMES ∪ catalog_credential_names()` .
- Validation (`POST /api/credentials/validate`) invokes `credential_manager.validate_api_key(provider, key, base_url?)` , which is intentionally non-raising — any non-200 response becomes `False` .
- Model listing (`POST /api/ai/models`) delegates to `ai_models.list_available_models` , which raises `ModelListError` on auth / transport failure.

Persistence Touchpoints

- `settings` table (SQLite) via `get_setting` / `set_setting` for every `/api/settings/*` endpoint. Schema version tracked in `SCHEMA_VERSION` .
- OS keychain via `credential_manager.store_credential` / `get_credential` / `delete_credential` (Python `keyring` , per ADQ-4). Service name is `resmon` (`APP_NAME`).
- Google Drive OAuth token persisted through `cloud_storage.authorize_google_drive` / `revoke_authorization` ; presence probed by `cloud_is_token_stored` and API reachability by `cloud_probe_api` .
- APScheduler job store (SQLAlchemy over SQLite, per ADQ-3) queried by `GET /api/scheduler/jobs` .
- OS-level service unit files (launchd / systemd / Task Scheduler) written by the `/api/service/install` path.

Execution Side Effects

- Saving `/api/settings/execution` updates the in-process `admission` controller's `max_concurrent_executions` and the scheduler's `routine_fire_queue_limit` , affecting subsequent manual and routine launches.
- `POST /api/settings/email/test` sends a real SMTP message using the stored `smtp_password` .
- `POST /api/cloud/backup` walks `REPORTS_DIR` and uploads files to Google Drive, returning the created folder's share link.
- `POST /api/service/install` and `/uninstall` mutate system-level service units and therefore whether routines can run while the app is closed.
- `POST /api/credentials/validate` and `/api/ai/models` issue a small number of outbound HTTPS requests to the provider's models endpoint.

Endpoints: Email Panel

- `GET /api/settings/email` — returns `{ smtp_server, smtp_port, smtp_username, smtp_from, smtp_to }` .
- `PUT /api/settings/email` — writes the same keys.
- `POST /api/settings/email/test` — constructs and sends a test message via the configured SMTP server using `smtp_password` from the keychain.
- `PUT /api/credentials/smtp_password` / `DELETE /api/credentials/smtp_password` — manages the keychain entry.

Endpoints: Cloud Account Panel

- `POST /api/cloud-auth/session` , `GET /api/cloud-auth/status` , `DELETE /api/cloud-auth/session` , `POST /api/cloud-auth/refresh` , `PUT /api/cloud-auth/sync` — JWKS-verified session lifecycle endpoints per IMPL-29 / 30. Present in the backend but not currently exercised by this panel because no hosted identity provider is wired.

Endpoints: Cloud Storage Panel

- `GET /api/cloud/status` — `{ is_linked, api_ok, api_reason }` .
- `POST /api/cloud/link` — runs `authorize_google_drive` (installed-app OAuth with `drive.file` scope).
- `POST /api/cloud/unlink` — runs `revoke_authorization` and discards the stored token.
- `GET / PUT /api/settings/cloud` — reads and writes `cloud_auto_backup` (and any other cloud-storage-scoped settings).

- `POST /api/cloud/backup` — walks `REPORTS_DIR` with `cloud_storage.upload_directory` and returns `{ uploaded, total_files, folder_name, web_view_link }`.

Endpoints: AI Panel

- `GET /api/settings/ai` / `PUT /api/settings/ai` — reads and writes the AI settings keys (see panel section above).
- `POST /api/credentials/validate` — `credential_manager.validate_api_key(provider, key, base_url?)`.
- `POST /api/ai/models` — `ai_models.list_available_models` using either an ephemeral key (pushed with `push_ephemeral` for the current request) or the stored credential.
- `PUT /api/credentials/{name}` / `DELETE /api/credentials/{name}` where `name ∈ AI_CREDENTIAL_NAMES` (`openai_api_key`, `anthropic_api_key`, `google_api_key`, `xai_api_key`, `meta_api_key`, `deepseek_api_key`, `alibaba_api_key`, `custom_llm_api_key`).

Endpoints: Storage Panel

- `GET /api/settings/storage` — returns `{ pdf_policy, txt_policy, archive_after_days, export_directory }`.
- `PUT /api/settings/storage` — writes the same keys; backend coerces numeric `archive_after_days` and validates policy values against `{save, archive, discard}`.

Endpoints: Notifications Panel

- `GET /api/settings/notifications` — returns `{ notify_manual, notify_automatic_mode }`.
- `PUT /api/settings/notifications` — writes the same two keys; `notify_automatic_mode` is constrained to `'all' | 'selected' | 'none'`.

Endpoints: Advanced Panel

- `GET /api/service/status`, `POST /api/service/install`, `POST /api/service/uninstall` — OS service-unit management (IMPL-26).
- `GET /api/settings/execution` / `PUT /api/settings/execution` — `{ max_concurrent_executions, routine_fire_queue_limit }` used by `admission` (IMPL-R1) and the scheduler's routine-fire queue (IMPL-R3 / R6).
- `GET /api/scheduler/jobs` — APScheduler job listing for diagnostics.
- `GET /api/health` — polled every 5 s to display PID, uptime, and version.
- **Danger Zone (Update 3 / `4_27_26`):**
  - `POST /api/admin/erase-ai-keys` — iterates `AI_CREDENTIAL_NAMES` and calls `credential_manager.delete_credential` for each.
  - `POST /api/admin/erase-repo-keys` — iterates `catalog_credential_names()` (the union of `requires_credential` repos in `repo_catalog.REPOSITORY_CATALOG`) and calls `delete_credential` for each.
  - `POST /api/admin/erase-configs` — `DELETE FROM saved_configurations`; cascades to routines via `linked_routine_id`; resets `sqlite_sequence` for the affected tables.
  - `POST /api/admin/erase-executions` — `DELETE FROM executions` (plus child progress / log rows) and `DELETE FROM sqlite_sequence WHERE name='executions'` so the auto-incremented `Execution #N` counter restarts at 1.
  - `POST /api/admin/erase-execution-data` — composite of `erase-configs` + `erase-executions`.
  - `POST /api/admin/erase-app-data` — composite of `erase-ai-keys` + `erase-repo-keys` + `erase-execution-data`. Non-AI Settings tabs (Email, Cloud Storage, Storage, Notifications, Advanced) are left alone.
  - `POST /api/admin/reset-settings` — wipes every row in the `settings` table back to defaults, deletes every AI / repo / SMTP credential, and clears the cached cloud-account email. Configs and executions are kept.
  - `POST /api/admin/factory-reset` — composite of `erase-app-data` + `reset-settings`; the resulting state is equivalent to a fresh install on this device.
  - `POST /api/admin/cloud/{erase-ai-keys,erase-repo-keys,erase-configs,erase-executions,erase-execution-data,erase-app-data,reset-s` — eight cloud-account mirrors that share the same handler shape as their local counterparts. They are scaffolding for the upcoming cloud-account feature; the UI renders the cloud column disabled until cloud sign-in lands.
  - All eight local endpoints (and their cloud mirrors) accept an empty body for the API-key wipes and `{ "confirm": "CONFIRM" }` for the six destructive data/settings actions; on success the frontend broadcasts on `configurationsBus`, `routinesBus`, and the `resmon:execution-completed` window event so every other page refetches.

Settings Page Info

Page Overview

Purpose

Primary User Flows

Inputs and Outputs

Known Constraints

Frontend

Route and Main Component

Child Components and Hooks

UI State Model

Key Interactions and Events

Error and Empty States

Panel: Email

Panel: Cloud Account

Panel: Cloud Storage

Panel: AI

Panel: Storage

Panel: Notifications

Panel: Advanced

Backend

API Endpoints

Request/Response Patterns

Persistence Touchpoints

Execution Side Effects

Endpoints: Email Panel

Endpoints: Cloud Account Panel

Endpoints: Cloud Storage Panel

Endpoints: AI Panel

Endpoints: Storage Panel

Endpoints: Notifications Panel

Endpoints: Advanced Panel